TRACES Developer Guide

Trajectory Research, Analysis, Correction, Exploration, and Simulation

Table of Contents

- 1. Purpose and Audience
- 2. Architecture Overview
- 3. Data Model
- 4. Module Map
- 5. Top-Level Modules
- 6. GUI Package
- 7. Data Import Package
- 8. UI Kit Package
- 9. Extending the Analysis Package
- 10. Plot Package and Interaction Handlers
- 11. Extending Lane-Level Anomaly Detection and Correction
- 12. Connecting Other Data Sources
- 13. Building and Packaging
- 14. Cross-Cutting Conventions
- Appendix: Related Documents

---

1. Purpose and Audience

This guide is for software engineers extending TRACES — adding file-format importers, data sources, analyses, correction modules, interactive plot modes, or lane-level anomaly types. It describes how the application's packages fit together and documents each extension point.

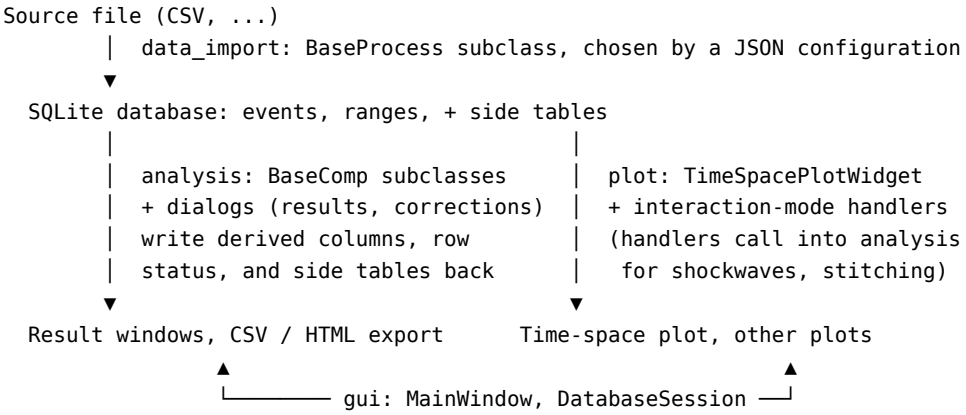
It doesn't restate material that has its own document:

- The JSON import configuration schema (column definitions, import functions, units, data dictionary) is in the *TRACES Configuration Guide*.
- End-user workflows and UI behavior are in the *TRACES User Guide*.

---

2. Architecture Overview

TRACES is a PyQt6 desktop application built around one SQLite database per opened trajectory dataset. Data is imported once into that database; every other feature reads it, and the analysis and correction modules write results and corrections back into it.



Package dependency order

The packages form a strict layering. Each depends only on packages earlier in this order, and never on a later one:

Order	Package	May import	Qt?
1	`utils` (module)	—	No
2	`data_import`	`utils`	No
3	`ui_kit`	`utils`	Yes
4	`analysis`	`utils`, `data_import`, `ui_kit`	Computation modules no; dialogs yes
5	`plot`	`utils`, `ui_kit`, `analysis`	Yes
6	`gui`	everything above	Yes
—	`traces.py`	anything	Yes

Consequences worth knowing before you add code:

- **Shared vocabulary lives low.** The row-status codes and SQL filter helpers are in `utils`, not in `analysis`, so that `plot`, `gui`, and `data_import` can use them. `analysis.base_comp` re-exports them for convenience.
- **Interaction handlers live in `plot`, computations in `analysis`.** `plot/handlers/shockwave_handler.py` calls `analysis.shockwave.ShockwaveComp`; `analysis` never imports `plot`. Anything a handler needs from an analysis must be reachable without importing Qt plot code.
- **`gui` wires everything together.** New features are implemented in the lower packages and then given a menu entry in `gui/main_window.py` (or a right-click entry in `plot/custom_view_box.py`).

Import style and package `__init__` files

- Within a top-level package, import with the relative form (`from ..base_comp import BaseComp`); across packages, use the absolute form (`from analysis.jerk import JerkCompDialog`).
- Every package's `__init__.py` re-exports its public names **lazily** (PEP 562): an `_EXPORTS` map of name → submodule, a module `__getattr__` that imports the submodule on first access, a `__dir__`, and an `if TYPE_CHECKING:` block repeating the imports for IDEs, type checkers, and PyInstaller's import scan. Importing a package therefore imports nothing — in particular, importing `analysis.lane_anomaly` on a worker thread does not import Qt.
- When you add a public class or function, add it to the package's `_EXPORTS` **and** its `TYPE_CHECKING` block. The two must list the same names.

---

3. Data Model

3.1 The `events` table

One row per `(event_id, veh_id, veh_time)`, the table's primary key (`WITHOUT ROWID`). The schema is **configuration-driven**: the import creates whatever columns the configuration declares. Only five columns are guaranteed: `event_id`, `veh_id`, `veh_time`, `lane_id`, `veh_dist_trav`. Every other column is optional, and any code that reads one must check for it first:

```
cols = utils.events_columns(conn)          # re-queries PRAGMA table_info every call
speed = "veh_speed" if "veh_speed" in cols else "NULL"
```

The schema also changes at run time — modules add columns as the user runs them — so call `events_columns()` when you need it rather than caching the result.

Columns the application adds:

Column	Added by	Notes
`imputed_status`	`analysis.base_comp.ensure_imputed_status_column()` — called by Spline Fitting, Gap Interpolation, Lane-Level Anomalies	Row provenance (§3.3). `TEXT NOT NULL DEFAULT ' '`. Migrates the legacy `is_spline_imputed` flag.
`traj_status`	`analysis.base_comp.ensure_traj_status_column()` — called by Trajectory Stitching	Trajectory status (§3.3). Also installs a trigger that copies a vehicle's status onto rows inserted later.
`calc_position`, `calc_speed`, `calc_acceleration`, `calc_jerk`	Spline Fitting	Listed in `analysis.base_comp.DERIVED_EVENTS_COLUMNS`.
`veh_speed_cal`, `veh_accel_cal`, `veh_jerk_cal`, `speed_error`, `accel_error`, `jerk_error`	Internal Consistency	Listed in `DERIVED_EVENTS_COLUMNS`.

All of these are reserved names that a configuration may not define; the list is `_INTERNAL_RESERVED` in `data_import/csv_process.py` (§7.2).

3.2 The `ranges` table

One row per event: `event_id` and `summary_json`. The import worker creates it after `import_data()` — no processor builds it — and its presence is how `validate_trajectory_db()` knows an import completed. `summary_json` is built by `data_import.db_utils.build_event_summary()`:

```
{"event_name": "run index: 1", "veh_time": [0, 140], "veh_dist_trav": [0, 812],
 "lane_id": [[1, 1], [2, 2], [3, -3]], "veh_id": [1, 2, 3]}
```

`lane_id` holds `[lane_id, orig_lane_id]` pairs; `veh_id` lists only active (not stitched-away) trajectories. A module that changes lane membership or adds or removes trajectories must call `db_utils.update_event_summary(conn, event_id)` inside its own transaction.

3.3 Row provenance and trajectory status

Row provenance is recorded **only** in `events.imputed_status`. The codes, their meaning, and their owners are registered in `utils.IMPUTED_CODES`:

Code	Constant	Owner (`utils.OWNER_*`)	Placeholder	Kinematics measured	Lane measured
`' '`	`IMPUTED_ORIGINAL`	`OWNER_IMPORT`	no	yes	yes
`'S'`	`IMPUTED_SPLINE`	`OWNER_SPLINE`	yes	no	no
`'A'`	`IMPUTED_AUTO`	`OWNER_GAP`	no	no	no
`'M'`	`IMPUTED_MANUAL`	`OWNER_GAP`	no	no	no
`'L'`	`IMPUTED_LANE`	`OWNER_LANE_ANOMALY`	no	yes	no

Rules every module follows:

- **Don't add status or flag columns to `events`.** Register a new code in `utils.IMPUTED_CODES` with its properties instead.
- **Select rows by property, never by code literal.** Use `utils.status_codes(owner=..., is_placeholder=..., kinematics_measured=..., lane_measured=...)` and `utils.status_in_clause(codes, negate=True)`, or the ready-made `utils.measured_rows_clause(cols)` (excludes placeholder rows). Prefer

excluding codes (`negate=True`) over listing them, so a code added later is treated as ordinary data instead of vanishing from queries.

- **Ownership.** A module changes a row's code only if the row is original or carries one of its own codes.
- **Save what you overwrite.** Before overwriting an existing row's values, call `analysis.base_comp.supersede_rows(conn, owner, keys)`; to undo, call `restore_superseded(conn, owner, event_id, ...)`. Records are keyed by owner, so changes by different modules stack and unwind in reverse order. `layer_source(conn, owner)` gives a derived table of the data as it reads with that owner's changes undone (Gap Interpolation's "measured data").
- **Keep your own records in side tables** (`gaps`, `stitches`, `lane_anomalies`, `lane_corrections`). If they are per-vehicle, add them to `analysis.base_comp.VEHICLE_KEYED_TABLES`, so `purge_vehicle_records()` cleans them up when a stitch undo deletes a trajectory.
- **When you add a code, audit the modules that read or own codes.** At present: `utils` (`measured_rows_clause`), `analysis.base_comp` (superseded helpers), `analysis.spline.spline_comp` and `spline_comp_dialog`, `analysis.gap_interpolation.gap_interpolation_comp`, `analysis.lane_anomaly.lane_correction` (`check_correctable`), `analysis.manual_stitching.stitching_comp`, `data_import.db_utils` (`_lane_pairs`), and the time-space plot's data query. Search for `status_codes(` and `IMPUTED_` to find others.

Trajectory status is the one other status column. `events.traj_status` is uniform per `(event_id, veh_id)`, owned by Manual Stitching (`utils.TRAJ_KEPT`, `TRAJ_REMOVED`, `TRAJ_STITCHED`), and filtered with `utils.active_rows_clause(cols)`, which excludes removed trajectories. Every query that reads trajectories for display or analysis applies both filters:

```
cols = events_columns(conn)
sql = (f"SELECT veh_time, veh_dist_trav FROM events WHERE event_id = ? AND veh_id = ?"
       f"{active_rows_clause(cols)}{measured_rows_clause(cols)} ORDER BY veh_time")
```

3.4 Side tables

Table	Owner	Contents
`superseded_points`	shared (`analysis.base_comp`)	Values and prior status of rows a module overwrote, keyed by `(event_id, veh_id, veh_time, owner)`. Older databases may hold Gap Interpolation records in `gap_superseded_points`, migrated on first write.
`gaps`	Gap Interpolation	One row per gap, with its spline parameters and status.
`stitches`	Manual Stitching	One row per stitch, including undone ones (`undone_at`).
`lane_anomalies`, `lane_corrections`	Lane-Level Anomalies	Detected anomalies, and the fixes applied to them.

3.5 Sampling interval and gaps

The single definition of a gap is in `utils`: `event_time_step(conn, event_id)` (the event's most common time step, falling back to `DEFAULT_TIME_STEP`), `gap_indices(t, step)`, `missing_samples(dt, step)`, and `segment_bounds(t, step)`. Use them rather than writing your own threshold.

---

4. Module Map

Location	Contents	Purpose
top level	traces.py, utils.py	Entry point; shared Qt-free helpers and the row-status vocabulary.
data_import/	base_process.py, csv_process.py, csv_algorithms.py, units.py, db_utils.py	Importing source files into the database; database validation and event summaries. Qt-free.
ui_kit/	brand.py, db_target_row.py, kinematic_stacked_plot_widget.py, box_plot_item.py, box_plot_grid_dialog.py, gap_point_editor.py, point_edit_utils.py, plot_style.py, plot_export.py, plot_view.js, toggle_switch.py, vertical_gradient_value.py	Reusable widgets, plot styling, and interactive HTML export.
analysis/	base_comp.py, base_analysis_dialog.py, base_analysis_worker.py, trajectory_utils.py	Base classes and shared helpers for every analysis.
analysis/jerk/, missing_data/, internal_consistency/, platoon/, spectral/	*_comp.py + dialogs	Noise analyses.
analysis/spline/	spline_comp.py + dialogs	Spline fitting; its math is shared with Gap Interpolation.
analysis/shockwave/	shockwave_comp.py + dialogs	Shockwave computation; the interaction is in plot/handlers/shockwave_handler.py.
analysis/advanced_statistics/	advanced_stats_comp.py, leader_follower.py, lane_changes.py, stat_registry.py + dialogs	Leader/follower and lane-change statistics.
analysis/gap_interpolation/	gap_interpolation_comp.py, gap_sort.py + dialogs	Gap detection and spline fill, with review.
analysis/manual_stitching/	stitching_comp.py, safety_rules.py, stitching_preview_dialog.py	Trajectory stitching.
analysis/lane_anomaly/	registry, store, correction, two anomaly types and their views, dialogs	Lane-level anomaly detection and correction; see §11.
plot/	time_space_plot_widget.py, custom_view_box.py, slider_view_box.py, settings, statistics, time-speed/accel, and vehicle-search dialogs	The time-space plot and its dialogs.
plot/handlers/	interaction_mode.py, gap_headway_handler.py, shockwave_handler.py, shockwave_result_panel.py, stitch_select_handler.py, plot_overlay_label.py	Interaction modes of the time-space plot; see §10.
gui/	main_window.py, database_session.py, home_widget.py, import, export, event selection, log viewer, and about dialogs	Application shell.
config/	ohio-*.json, tgsim-*.json	Bundled import configurations.
assets/	traces_icon.svg, traces_logo.svg, make_icons.py	Artwork; build-time icon generation (§13).

5. Top-Level Modules

Module	Purpose
`traces.py`	Entry point. Configures logging, calls `utils.initialize_user_config()` (seeds the user `config` folder from the bundled configurations on first run), creates the `QApplication`, applies `ui_kit.brand.apply_application_branding()`, and shows `gui.main_window.MainWindow`.
`utils.py`	Qt-free helpers shared by every package: `get_resource_path()` (bundled read-only resources; works in PyInstaller builds), `get_user_data_path()` (writable user data: `%APPDATA%\TRACES` when frozen, the working directory in development), `initialize_user_config()`, the "remember last folder" pair `last_dialog_dir()` / `remember_dialog_path()`, `default_save_stem()`, `events_columns()`, the row-status vocabulary and filters (§3.3), and the sampling helpers (§3.5).

User data written by the application: `traces.conf` (window geometry and recent files), `traces.log` (the import log shown by the System Log Viewer), and the `config` folder.

6. GUI Package

`gui` is the application shell. It's not normally where features are implemented — they're implemented in the lower packages and wired in here.

Component	Purpose
`main_window.py` — `MainWindow`	Owns the menu bar (File / View / Data / Analysis / Mode), recent files, and the central widget (home page or plot). Creates the import, export, analysis, and correction dialogs on demand. Data, Analysis, and Mode stay hidden until a database is open.
`database_session.py` — `DatabaseSession`	Owns the open connection: `open()`, `close()`, `ensure_write_target()`, `copy_and_switch()`. Emits `database_switched(new_conn, old_conn, new_path)` when changes are redirected to a copy. See §6.1.
`home_widget.py` — `HomeWidget`	Start screen (Open / Import cards, recent files).
`import_dialog.py` — `ImportDialog`	Discovers configurations and drives an import processor on a worker thread (§7).
`export_dialog.py` — `ExportDialog`	Export All / Export Selected.
`event_selection_dialog.py` — `EventSelectionDialog`	Multi-event picker used by Spline Fitting and Export Analysis.
`log_viewer_dialog.py` — `LogViewerDialog`	Tails `traces.log`.
`about_dialog.py` — `AboutDialog`	About box.

6.1 Write targets

Modules that **change** the database (Gap Interpolation, Lane-Level Anomalies, Trajectory Stitching) don't write to the open connection directly. They show a `ui_kit.db_target_row.DbTargetRow` ("Write changes to: [path] Browse…") and, before their first write, call a **resolver**:

```
resolver(parent: QWidget, requested_path: str) -> sqlite3.Connection | None
```

`MainWindow` passes `DatabaseSession.ensure_write_target` as the resolver (`resolve_target=` argument). It shows the one-time Write Changes prompt for the open file, or copies the database to another path and switches the session to it; `None` means the user cancelled. `in_place_resolver(conn)` is the fallback when a dialog is created without one. After resolving, the dialog writes through the returned connection and calls `DbTargetRow.set_connection()` so the row shows the file actually written.

When the session switches to a copy, `MainWindow` swaps the plot's connection (`TimeSpacePlotWidget.set_db_connection()`) and closes visible dialogs still holding the old connection — except dialogs with the class attribute `KEEPS_OPEN_ON_DB_SWITCH = True` (the Gap Interpolation and Lane-Level Anomalies configuration dialogs), which requested the switch and continue on the new connection.

Modules that only **add result columns** (Spline Fitting, Internal Consistency) write to the open connection without a resolver.

6.2 Telling the plot that data changed

A correction window that changes `events` emits `data_changed(event_id)` once, when it closes. `MainWindow._on_event_data_changed()` reloads the time-space plot if it shows that event, keeping its view. Because the review windows outlive the configuration dialogs that open them, those dialogs emit `review_opened(dialog)` so `MainWindow` can connect to the review window directly — see `_run_gap_interpolation()` and `_run_lane_anomalies()`.

---

7. Data Import Package

7.1 The `BaseProcess` contract

An import processor owns everything specific to one source format. The orchestration around it is fixed:

```
class BaseProcess(ABC):
    def check_configuration(self, config: Dict[str, Any], log_fn: LogFn) -> int
    def column_defs(self) -> Dict[str, str]
    def check_file(self, input_path: str, worker: Any) -> int
    def import_data(self, input_path: str, db_connection: sqlite3.Connection, worker: Any) -> int
```

Stage	Runs on	Responsibility
`check_configuration(config, log_fn)`	GUI thread	Validate the JSON configuration and keep what later stages need. Must not touch the database or the input. Returns an error count (0 = valid). `log_fn(message, is_html)`; messages starting `"Info:"` are suppressed during the startup scan.
`column_defs()`	worker	Ordered `{column: "INTEGER" | "REAL" | "TEXT"}`, including the five required columns.

Stage	Runs on	Responsibility
`check_file(input_path, worker)`	worker	Validate the whole input before anything is written. Returns an error count; any error stops the import.
`import_data(input_path, db_connection, worker)`	worker	Populate `events`. Return 0 on success, non-zero on failure.

`worker` provides `log_signal(str, bool)`, `progress_signal(int)` (0–100), and `interrupt_flag` — check it in long loops so Interrupt responds.

Orchestration (in `ImportDialog` and `_ImportWorkerThread`, not in processors):

1. When the dialog opens, it reads every `.json` file in `get_user_data_path("config")`, resolves `"class"` with `importlib`, checks it subclasses `BaseProcess`, and validates the configuration on a throwaway instance. Valid configurations become file types, labeled by file stem.
2. At import start it creates a **fresh** processor instance (no state carries between runs), validates again with full logging, and hands it to the worker.
3. The worker removes any existing output file, creates the `events` table from `column_defs()` (primary key on `event_id, veh_id, veh_time`; `INSERT OR IGNORE` drops duplicates), then calls `check_file()` and `import_data()`.
4. The worker builds `ranges` with `db_utils.build_event_summary()`. A failed or interrupted import removes the output file.

7.2 `CsvProcess` — the reference implementation

`CsvProcess` implements the stages for CSV files described by a `col_defs` configuration (fully documented in the *TRACES Configuration Guide*). Points that matter when you modify it or write a sibling:

- **Validation constants** at the top of `csv_process.py`: `_REQUIRED_COLS`, `_INTERNAL_RESERVED` and `_INTERNAL_RESERVED_PREFIXES` (names the application creates — keep in step with §3.1 and `DERIVED_EVENTS_COLUMNS`), and the `_FORCED_*_COLS` sets (storage types of Data Dictionary columns). `data_import` may not import `analysis`, so the reserved list is maintained by hand.
- **Units.** `units.Units.SYSTEM_UNITS` maps each Data Dictionary column to its stored unit (`""` = unitless, including geographic degrees); `_FACTORS` is a table of multipliers. Add a conversion by adding both directions to `_FACTORS`.
- **Records with no key value** are skipped and listed in the log (`_KEY_COLS`, `_MAX_SKIP_MESSAGES`); any other database error ends the import.

7.3 Adding an import function

Import functions are the public methods of `CsvAlgorithms` (`csv_algorithms.py`). `CsvProcess` discovers them by introspection, so a new function needs no change elsewhere:

```
@source_cols(lambda parms: [parms[0], parms[1]])      # which parms are CSV column indices
def ratio(self, row: List[str], col_def: Dict[str, Any]) -> Optional[float]:
    """parms[0] / parms[1]; null when either is empty or the divisor is 0."""
    a, b = (row[i] for i in col_def["parms"][:2])
    try:
        return _convert(float(a) / float(b), col_def.get("type", "float"))
    except (ValueError, ZeroDivisionError):
        return None
```

- Signature `(self, row, col_def)`; return the value to store, or `None` for null. Unit conversion is applied afterwards by `CsvProcess`.

- `@source_cols(extractor)` maps `parms` to the entries that are column indices. They are bounds-checked against every CSV row before import. Let the extractor raise `IndexError` when `parms` is too short — that becomes a configuration error. Without the decorator, every integer parameter counts as a column index.
- Don't raise for unusable values; return `None`.
- Helpers must start with `_` so they aren't offered as functions. A `CsvAlgorithms` instance is created per import, so a function may keep state across rows (as `group` does).
- Document the function in the *TRACES Configuration Guide*, section "Supported Import Functions".

7.4 Adding a file-format importer

1. Subclass `BaseProcess` (in `data_import/` or your own package) and implement the four stages. Keep configuration state on `self` in `check_configuration()` — it's the only stage that receives the configuration.
2. Return the five required columns from `column_defs()`, and reject the reserved names (reuse `csv_process._INTERNAL_RESERVED`, or move it to a shared module).
3. Reuse `Units` for unit-aware columns.
4. Write a configuration with `"class": "your.module.YourProcess"` and `"file_ext": "<ext>"`, plus your own keys, and put it in the user `config` folder (or in `config/` to bundle it).
5. No change is needed in `ImportDialog` or `MainWindow` for a local file format. Sources that aren't local files do need changes there — see §12.
6. Make sure the build bundles the module (§13) and document the configuration keys in the *TRACES Configuration Guide*.

---

8. UI Kit Package

Reuse these instead of building new versions; they keep the application's plots consistent.

Component	Use it for
`kinematic_stacked_plot_widget.KinematicStackedPlotWidget`	Distance / speed / acceleration / jerk plots on one linked time axis: number keys and scrolling for plots in view, hover cursor and values, legend toggles, `export_spec()` for HTML export. Register data with `add_series(name, role, linked=...)` and `set_data(series, time, {key: values})`; add markers with `add_vlines()` / `add_item()`.
`plot_style`	`Role` (MEASURED, DERIVED, PREVIEW, SEAM, AUTO_FILL, BOUNDARY, SPLIT, CROSSHAIR) and `ROLE_STYLES`: pens, dashes, colors, and z-order by meaning. `role_pen()`, `region_brush(name)` (`"auto"`, `"manual"`, `"unfilled"`, …), `apply_plot_style(plot_widget)`, `KINEMATIC_COLORS`, `PROPOSAL_COLOR`.
`plot_export`	Self-contained interactive HTML pages: `build_plot_html(title, plots)`, with `plot_spec()`, `curve_item()`, `vline_item()`, `region_item()`, `box_item()`. The viewer script `plot_view.js` is read with `importlib.resources`, so `ui_kit` must stay a regular package.
`box_plot_grid_dialog.BoxPlotGridDialog`	A grid of box plots with hover statistics, outlier labels, and HTML save. Describe each plot as a `BoxPlotPanel(key, title, [(label, value), ...])`. Used by Jerk, Missing Data, and Advanced Statistics.

Component	Use it for
`db_target_row.DbTargetRow`	The "Write changes to" row and the resolver contract (§6.1).
`gap_point_editor.GapPointEditor`, `point_edit_utils`	Click / Ctrl+click / drag point editing on one plot (Gap Interpolation's Add Points).
`brand`	`APP_NAME`, `APP_VERSION`, colors, `logo_pixmap()`, `app_icon()`.
`toggle_switch.ToggleSwitch`, `vertical_gradient_value.VerticalGradientValue`	The plot's Show Speed switch and gradient legend.

---

9. Extending the Analysis Package

9.1 Base classes

`BaseComp` (`analysis/base_comp.py`) is the Qt-free computation interface:

```
class BaseComp(ABC):
    OPTIONS_SCHEMA: Dict[str, Any] = {}          # option key -> default

    def comp(self, db_connection, options=None) -> List[Any]                    # every event in
     ↪ ranges
    def comp_event(self, db_connection, event_id, options=None) -> List[Any]     # comp_series(...,
     ↪ None, ...)
    def comp_series(self, db_connection, event_id, veh_ids, options=None) -> List[Any]   # override
     ↪ this
```

- Override only `comp_series()`. `veh_ids=None` means every active vehicle of the event (the menu path); an iterable scopes the analysis (the right-click path).
- Parameters travel in the `options` dict; declare keys and defaults in `OPTIONS_SCHEMA`, and merge with `self._resolve_options(options)`. Adding a parameter never changes a signature.
- Helpers: `_events_columns(conn)` and `_event_veh_ids(conn, event_id)` (active vehicles only).
- Return a list of result objects; a `@dataclass` per (event, vehicle) is the convention.
- Read with `active_rows_clause()` and `measured_rows_clause()` (§3.3). Don't write to the database from a comp class — the dialog persists results on the main thread.

`BaseAnalysisDialog` (`analysis/base_analysis_dialog.py`) is the standard parameter dialog: a parameter form, a progress bar, and OK (or Save in export mode) / Cancel, with the computation on a worker thread. Subclasses implement:

```
def _build_parameter_form(self) -> QFormLayout      # widgets; keep references on self
def _current_params(self) -> dict                   # widget values -> options dict
def _create_worker(self, db_path, event_ids, veh_ids, params, dest_path) -> BaseAnalysisWorker
def _csv_export_label(self) -> str                  # window title and export messages
def _show_results(self, results: list) -> None      # open the results window
# optional:
def _set_controls_enabled(self, enabled: bool)      # call super() first
def _progress_maximum(self, event_ids, veh_ids, params) -> int   # default: one tick per event
```

Constructor: `(parent, db_connection, event_ids=None, veh_ids=None, export_mode=False)`. The dialog records `self._veh_ids` (None for whole-event runs) and `self._export_event_ids`.

Path	Created with	Button	On finish
Menu (whole event)	`event_ids=[current]`	OK	`_show_results(results)`

Path	Created with	Button	On finish
Right-click (scoped)	`event_ids=[current], veh_ids=[...]`	OK	`_show_results(results)`; branch on `self._veh_ids is not None`
Export Analysis	`event_ids=[...], export_mode=True`	Save	Asks for a CSV path; the worker calls `write_csv(results, path)`; a completion message is shown

`BaseAnalysisWorker` (`analysis/base_analysis_worker.py`) is a `QObject` the dialog moves to a `QThread` — never move it yourself. It opens **its own** SQLite connection from `db_path` (connections can't cross threads), then calls

```
run_for_event(conn, event_id, veh_ids, params, progress_callback) -> list
```

once per event, concatenates the results, optionally calls `write_csv(results, dest_path)`, and emits `finished(list)` or `error(str)`. Call `progress_callback()` once per tick that `_progress_maximum()` promised.

9.2 Choosing a pattern

Pattern	Examples	Use when
Standard analysis — `BaseComp` + `BaseAnalysisDialog` + results window	Jerk, Missing Data, Internal Consistency, Advanced Statistics, Spline Fitting (Data menu)	The analysis runs over events or selections and shows or exports results. The default choice.
Correction module — standard dialog + write target + modeless review window	Gap Interpolation, Lane-Level Anomalies	The module changes trajectory data and the user reviews items one by one.
Synchronous preview — plain `QDialog`, no worker	Spline Fitting preview (`SplineTestingDialog`)	One trajectory, fast enough to recompute on a button press, writes nothing.
Interactive — computation in `analysis`, interaction in a `plot` handler	Shockwave, Trajectory Stitching	The result is built by the user on the time-space plot (§10).

9.3 Worked example: Jerk Analysis

1. **`analysis/jerk/jerk_comp.py`** — `JerkComp(BaseComp)` with `OPTIONS_SCHEMA` and `comp_series()`, returning `JerkResult` dataclasses.
2. **`analysis/jerk/jerk_comp_dialog.py`** — `JerkCompDialog(BaseAnalysisDialog)`:
   - `_create_worker()` returns `BaseAnalysisWorker(..., run_for_event=self._run_for_event, write_csv=self._write_csv)`;
   - `_run_for_event()` calls `comp_series()` or `comp_event()` depending on `veh_ids`, then `progress_callback()`;
   - `_show_results()` opens `JerkResultsTableDialog` for a right-click run, or `JerkBoxPlotDialog` (a `BoxPlotGridDialog`) for a whole-event run.
3. **`analysis/jerk/__init__.py`** — `JerkComp`, `JerkResult`, and the dialogs in `_EXPORTS` and `TYPE_CHECKING`.
4. **`gui/main_window.py`** — **Analysis → Noise → Jerk Analysis** (`_run_jerk_analysis`, whole event) and **File → Export → Export Analysis → Jerk Analysis** (`_export_jerk_analysis`: an `EventSelectionDialog`, then the dialog with `export_mode=True`).
5. **`plot/custom_view_box.py`** — the right-click **Noise Analysis → Jerk Analysis** entry in `_context_menu_registry`, calling a method that creates the dialog with the context's vehicles.

9.4 Adding an analysis

1. Create `analysis/<name>/` with `<name>_comp.py` (Qt-free `BaseComp` subclass) and dialog modules; give it an `__init__.py` in the house style (§2).
2. Implement `comp_series()`, reading with `active_rows_clause()` / `measured_rows_clause()` and gating optional columns on `events_columns()`.
3. Subclass `BaseAnalysisDialog`; build results windows from `ui_kit` (`BoxPlotGridDialog`, `KinematicStackedPlotWidget`, `plot_export`).
4. Wire it into `gui/main_window.py` — a `QAction` under **Analysis** (or **Data** for modules that change data), and under **Export Analysis** if a CSV export makes sense.
5. For right-click access, add a registry entry in `plot/custom_view_box.py` (§10.4).
6. If it writes per-row result columns:
   - add them with an idempotent `ALTER TABLE events ADD COLUMN` helper (see `InternalConsistencyComp.ensure_schema`);
   - list them in `analysis.base_comp.DERIVED_EVENTS_COLUMNS` (Trajectory Stitching blanks these on stitched rows);
   - add them to `_INTERNAL_RESERVED` in `data_import/csv_process.py` and to the *TRACES Configuration Guide*, section "Columns Created by the Application".
7. If it changes rows or keeps records, follow §3.3: register codes, supersede before overwriting, use side tables, add per-vehicle tables to `VEHICLE_KEYED_TABLES`, and audit the code readers.
8. If it changes data, use a write target (§6.1), set `KEEPS_OPEN_ON_DB_SWITCH` if the dialog should survive a switch to a copy, and emit `data_changed(event_id)` when done (§6.2).
9. Document it in the *TRACES User Guide*.

9.5 Smaller extension points

To add	Edit
A statistic in the Advanced Statistics table	`analysis/advanced_statistics/stat_registry.py` — add a `ScalarStat` (one value per vehicle) or `StatGroup` (mean / min / max / Q1 / Q3 from samples; offered in Generate Plots) to `ADVANCED_STATISTICS`. The table, CSV, summary row, and plot list are built from it.
A stitching safety check	`analysis/manual_stitching/safety_rules.py` — a pure function `SafetyContext -> Optional[str]` appended to `SAFETY_RULES`. Rules warn, never block.
A different gap display order	`analysis/gap_interpolation/gap_sort.py` — `sort_gaps()` / `default_gap_sort_key()`.
A lane-level anomaly type	§11.

---

10. Plot Package and Interaction Handlers

10.1 `TimeSpacePlotWidget`, `CustomViewBox`, and `SliderViewBox`

`TimeSpacePlotWidget` (`plot/time_space_plot_widget.py`) owns the time-space plot: the scatter plot, the time and distance range sliders (the single source of truth for the visible range), the lane list, the Show Speed switch, the Mode menu's actions, and the loaded data.

- **`self._series`** — `{veh_id: dict}` with `veh_time`, `veh_dist_trav`, `veh_speed` (numpy arrays for the lane shown), `plot_item`, `color`, and `segments`. Hit-test against this; don't query the database on mouse events. `find_vehicle_at(x, y)` and `_nearest_point(x, y)` are the shared hit tests.
- **`self._selected_series`** — the selected vehicle IDs; call `_update_plot_colors()` after changing it.

- **`self._update_plot_data()`** — reloads the lane from the database (also after a lane, event, or mode change).
- **`self._db_connection`**, **`self._current_event_id`**, **`self._interaction_mode`**, **`self._active_handler`**.

`CustomViewBox` (`plot/custom_view_box.py`) is the plot's view box. It turns mouse wheel gestures into slider changes, implements the default Select behavior (click to toggle, drag to box-select, Shift+drag to deselect), and builds the right-click menu from `_context_menu_registry`. `SliderViewBox` (`plot/slider_view_box.py`) is the slider-driven view box used by the Time-Speed and Time-Accel plots.

10.2 How handlers interact with the plot

Each interaction mode is an `InteractionModeHandler` (`plot/handlers/interaction_mode.py`). The widget keeps one active instance in `_active_handler`, and the widget and view box forward events to it instead of branching on the mode:

```
class InteractionModeHandler:
    # Lifecycle (widget = TimeSpacePlotWidget)
    def on_enter(self, widget)                # mode became active
    def on_exit(self, widget)                 # switching away: remove overlays, restore state
    def on_before_data_reload(self, widget)   # start of _update_plot_data, before the plot is
     ↪ cleared
    def on_plot_cleared(self, widget)         # after scatter_plot.clear(): drop cached item
     ↪ references
    def on_data_reloaded(self, widget)        # after new data is loaded: redraw overlays

    # From CustomViewBox (view_box.parent_widget is the widget); return True to consume
    def on_mouse_press(self, view_box, ev) -> bool
    def on_mouse_move(self, view_box, ev) -> bool      # True suppresses the selection box
    def on_mouse_release(self, view_box, ev) -> bool   # True suppresses click / box select
    def on_right_click(self, view_box, ev) -> bool     # True suppresses the context menu

    # From TimeSpacePlotWidget
    def on_hover(self, widget, pos, closest_data, min_dist) -> bool   # True suppresses the tooltip
    def on_key_press(self, widget, event) -> bool      # Escape, Return, and Enter only
    def on_slope_changed(self, widget, value)          # Shockwave sliders
    def on_vicinity_changed(self, widget, value)
    def wants_spline_position_data(self, widget) -> bool   # plot calc_position instead of
     ↪ veh_dist_trav
```

Every hook defaults to "not handled", so Select mode is a plain `InteractionModeHandler()`. A hook that returns `True` must call `ev.accept()` itself.

The sequence of a mode change (`_on_mode_changed(mode, handler=None)`):

1. `on_exit()` on the old handler.
2. The new handler is the one passed in, or `_HANDLER_FACTORIES[mode]()`.
3. `on_enter()` on the new handler; Vehicle Search is enabled only in Select mode.
4. `_update_plot_data()` reloads the plot, which calls `on_before_data_reload()`, `on_plot_cleared()`, `wants_spline_position_data()`, and `on_data_reloaded()` in turn.

The same reload sequence runs on every lane or event change, so a handler that draws overlays keeps **keys** (vehicle, time) rather than graphics items, and redraws from them in `on_data_reloaded()`.

10.3 Existing handlers

Handler	Mode	Pattern it shows
`GapHeadwayModeHandler`	Gap Headway	The simplest complete handler: hover overlay (`on_hover`), lock on click (`on_mouse_release`), suppressed box select (`on_mouse_move`), Escape clears (`on_key_press`), state kept as keys and redrawn in `on_data_reloaded`.
`ShockwaveSemiAutoModeHandler`	Shockwave	A configured handler: built by `_on_shockwave_action_triggered()` from `ShockwaveConfigDialog` and passed in, since its constructor needs arguments. Uses the slider hooks, Enter/Escape, Ctrl+click editing, `on_right_click` for deleting, and `wants_spline_position_data`. Calls `analysis.shockwave.ShockwaveComp` for inflection points and fits; results show in `ShockwaveResultPanel`. Manual and Automatic modes exist in `InteractionMode` but have no handler yet.
`StitchSelectModeHandler`	Trajectory Stitching	A mode entered from the right-click menu, not the Mode menu: `start_stitch_selection(veh_id)` / `end_stitch_selection()`. Draws halos, shows a `PlotOverlayLabel` banner, opens `analysis.manual_stitching.StitchingPreviewDialog`, and applies the result with `refresh_after_stitch_change()` without moving the view.

10.4 The right-click menu registry

`CustomViewBox._context_menu_registry` is a list of entries, rendered for the current context — `'A'` (All), `'S'` (Selected), or `'O'` (one Series):

```
{'filter': 'ASO', 'label': 'Noise Analysis', 'submenu': [
    {'label': 'Jerk Analysis', 'callback': self._run_jerk_analysis_rightclick},
    {'label': 'Missing Data', 'callback': self._run_missing_data_analysis_rightclick},
    {'label': 'Internal Consistency', 'callback': self._run_ic_rightclick},
    {'label': 'Platoon Consistency', 'callback': self._run_platoon_rightclick},
]},
{'filter': 'ASO', 'label': 'Stitching', 'submenu': [
    {'label': 'Basic Stitching', 'callback': self._start_stitching,
     'single_only': True, 'select_mode_only': True,
     'single_tooltip': "Select exactly one trajectory to stitch."},
]},
```

Keys: `filter` (contexts in which the item is enabled), `label`, `callback` (`None` = disabled), `submenu`, `single_only` (exactly one target vehicle), `select_mode_only`, `requires_column` (an `events` column), `single_tooltip`, `disabled_tooltip`. Disabled items get an explanatory tooltip. In a callback, `self._get_target_vehicle_ids()` returns the context's vehicles and `self._get_statistics_mode()` returns `"All"`, `"Selected"`, or `"Series"`.

10.5 Adding an interaction mode

1. Add a member to `InteractionMode`.
2. Create a handler in `plot/handlers/`, overriding only the hooks you need, and export it from `plot/handlers/__init__.py`.
3. Register it in `TimeSpacePlotWidget._HANDLER_FACTORIES`, or — if it needs constructor arguments — build it where the mode is started and call `_on_mode_changed(mode, handler=instance)`.
4. Start it: add a checkable action in `_setup_mode_menu()` and to `_mode_actions` (the action to re-check if a configuration dialog is cancelled), or start it from a right-click entry as stitching does.
5. Keep computation in `analysis` and call it from the handler.
6. Restore everything you changed in `on_exit()` (overlays, banners, the selection, slider visibility).
7. If the mode plots a different position column, extend `wants_spline_position_data()` rather than reading data yourself.

---

11. Extending Lane-Level Anomaly Detection and Correction

`analysis/lane_anomaly` is built so that anomaly types can be added without changing its dialogs. Each type has a Qt-free half, which detects anomalies and applies fixes, and a Qt half, which provides its controls. A registry joins them.

11.1 How the pieces fit

Module	Role
`anomaly_registry.py`	`AnomalyType` (the Qt-free half), `register_anomaly_type()`, `anomaly_types()`, `anomaly_type(key)`, `_BUILTIN_MODULES`.
`anomaly_ui.py`	`AnomalyTypeUI` (the Qt half), `ConfigRow`, `AnomalyPlot`, `Preview`, and the fix panels `FixPanel` → `SegmentFixPanel` → `VehicleLanesFixPanel`.
`lane_anomaly_store.py`	`LaneAnomaly` and `CorrectionRecord`, the `lane_anomalies` / `lane_corrections` tables, `replace_detected()` (rerun policy: unfixed anomalies of the rerun types are replaced, fixed ones kept).
`lane_correction.py`	`LaneMove`, `apply_lane_reassignments()`, `undo_correction()`, `check_correctable()`, and read helpers (`vehicle_rows`, `lane_traffic`, `event_lanes`, `neighbor_lanes`).
`lane_anomaly_comp.py`	`LaneAnomalyComp(BaseComp)`: runs the enabled types' `detect()` on the worker thread.
`anomaly_config_dialog.py`	Select Anomalies to Identify: one `ConfigRow` per registered type.
`anomaly_review_dialog.py`	The review panel: a count line and button per type, the Selected Anomaly list, `AnomalyPlot`, the type's fix panel, Apply / Undo.
`proximity.py` + `proximity_view.py`, `rapid_lane_change.py` + `rapid_lane_change_view.py`	The two built-in types — use them as templates.

The flow: the configuration dialog asks each type's UI for a `ConfigRow`; `LaneAnomalyComp` calls each checked type's `detect()`; the dialog stores the results with `replace_detected()`; the review panel shows them with the type's `describe()`, `title()`, and fix panel; **Apply** passes the panel's `pending_fix()` to the type's `apply_fix()`, and **Undo** calls `undo_fix()`.

Every built-in fix is a **lane reassignment** — a set of `LaneMove(veh_id, start_time, end_time, to_lane)` — because a lane-level correction only changes `lane_id`. `apply_lane_reassignments()` then does everything in one transaction: checks that every affected row is an active, original row (`check_correctable()`), saves the previous lane and status with `supersede_rows(..., OWNER_LANE_ANOMALY, ...)`, writes the new `lane_id` with `IMPUTED_LANE`, records one `lane_corrections` row per vehicle, marks the anomaly fixed, and refreshes the event summary. `undo_correction()` reverses all of it. A new type whose fix is a lane reassignment inherits all of this.

11.2 Step 1 — the detection class

Create a module in `analysis/lane_anomaly/` (or any importable package):

```
from typing import Any, Dict, Iterable, List, Optional
import sqlite3

from utils import active_rows_clause, events_columns, measured_rows_clause
from .anomaly_registry import AnomalyType, register_anomaly_type
from .lane_anomaly_store import LaneAnomaly

LANE_GHOST_KEY = "lane_ghost"


class LaneGhostType(AnomalyType):
    KEY = LANE_GHOST_KEY                  # stored in lane_anomalies.anomaly_type; never change it
    LABEL = "Lane Ghost"                  # dialog row and review button text
    UI_MODULE = ".lane_ghost_view"        # module defining ANOMALY_UI, relative to this package
    OPTIONS_SCHEMA: Dict[str, Any] = {"max_duration_s": 1.0}

    def detect(self, conn: sqlite3.Connection, event_id: int,
               veh_ids: Optional[Iterable[int]], options: Dict[str, Any],
               time_step: float) -> List[LaneAnomaly]:
        options = self.resolve_options(options)
        cols = events_columns(conn)
        rows = conn.execute(
            "SELECT veh_id, veh_time, lane_id FROM events WHERE event_id = ?"
            f"{active_rows_clause(cols)}{measured_rows_clause(cols)} "
            "ORDER BY veh_id, veh_time", (event_id,)).fetchall()
        anomalies = []
        # ... find short excursions into another lane ...
        #     anomalies.append(LaneAnomaly(
        #         event_id=event_id, anomaly_type=LANE_GHOST_KEY, veh_id=v,
        #         start_time=t0, end_time=t1, lane_ids=[home, other],
        #         params={"max_duration_s": options["max_duration_s"]},
        #         metrics={"samples": n}))
        return anomalies


register_anomaly_type(LaneGhostType())
```

Rules for `detect()`:

- **Read-only, Qt-free.** It runs on the worker thread with the worker's connection.
- **Active, measured rows only** — apply `active_rows_clause()` and `measured_rows_clause()`.
- **Honor `veh_ids`**: when it isn't `None`, return only anomalies involving those vehicles.
- **Use the grid** `time_step` (the event's sampling interval) to match timestamps across vehicles, as Proximity does with `round(veh_time / time_step)`.

- **Return `LaneAnomaly` objects with `anomaly_id=None`.** Set `veh_id` (and `other_veh_id` for a two-vehicle anomaly), `start_time` / `end_time` (the span whose lane assignment is in question), `lane_ids`, `params` (the options used), and `metrics` (anything the UI will show; stored as JSON).

Optional overrides on the type:

Method	Default	Override to
`describe(anomaly)`	`#id Veh a & Veh b t0–t1 s (lane …)`	Add a metric to the Selected Anomaly list (Proximity adds the minimum spacing).
`fix_vehicles(anomaly)`	the anomaly's vehicles	Restrict or order the vehicles that can be moved.
`fix_target_lanes(conn, anomaly)`	every lane of the event	Restrict the lanes offered.
`default_target_lane(conn, anomaly, veh_id)`	`None` ("Keep")	Preselect a lane (Rapid Lane Change preselects the lane holding more samples).
`apply_fix` / `undo_fix` / `describe_fix`	lane reassignment	A fix that isn't a lane reassignment — see §11.5.
`FIX_TYPE`	`"lane_reassign"`	Label this type's rows in `lane_corrections.fix_type`.

11.3 Step 2 — the UI module

Create the module named in `UI_MODULE`. It must define `ANOMALY_UI`, an instance of an `AnomalyTypeUI` subclass constructed with the type's key:

```
from PyQt6.QtWidgets import QDoubleSpinBox, QWidget
from .anomaly_ui import AnomalyTypeUI, ConfigRow
from .lane_anomaly_store import LaneAnomaly
from .lane_ghost import LANE_GHOST_KEY


class LaneGhostUI(AnomalyTypeUI):
    BUTTON_TEXT = "Lane Ghost Anomalies"

    def create_config_row(self, parent: QWidget) -> ConfigRow:
        spin = QDoubleSpinBox()
        spin.setRange(0.1, 10.0)
        spin.setValue(float(self.type.OPTIONS_SCHEMA["max_duration_s"]))
        return ConfigRow(LANE_GHOST_KEY, "Lane Ghost", "max_duration_s",
                         "Max Excursion", "s", spin, parent)

    def title(self, anomaly: LaneAnomaly) -> str:
        return f"Anomaly #{anomaly.anomaly_id} -- vehicle {anomaly.veh_id}"


ANOMALY_UI = LaneGhostUI(LANE_GHOST_KEY)
```

`AnomalyTypeUI` members:

Member	Required	Purpose
`BUTTON_TEXT`	yes	The type's button in the review panel.

Member	Required	Purpose
`create_config_row(parent)`	yes	The type's row in Select Anomalies to Identify. `ConfigRow(key, checkbox_text, param_key, param_label, unit, spin)` gives a checkbox and one parameter; `options()` returns `{param_key: value}`, which is merged over `OPTIONS_SCHEMA`. For several parameters, subclass `ConfigRow` and override `options()` and `set_controls_enabled()`.
`create_fix_panel(parent, conn, event_id)`	no	The fix panel. Default: `VehicleLanesFixPanel` (Start / End, and a "Move to" choice per vehicle).
`title(anomaly)`, `vehicle_label(anomaly, veh_id, index)`	no	Text the review plot shows for the anomaly and in its legend.
`count_text(n, n_fixed)`	no	The "N detected (M fixed)" line.
`type`	—	The type's `AnomalyType`, from the registry.

The UI module is imported only when a dialog first needs it (`AnomalyType.ui()`), so detection never imports Qt.

11.4 Step 3 — the fix panel

Choose the lowest level that fits:

- **Default `VehicleLanesFixPanel`** — nothing to write. Uses `fix_vehicles()`, `fix_target_lanes()`, and `default_target_lane()` from the type.
- **Subclass `SegmentFixPanel`** — keeps the Start / End segment editor, the preview, and the summary line, and lets you choose the lanes differently. Implement three methods, as `RapidLaneChangeFixPanel` does:

  ```
  def _build_choices(self, layout: QVBoxLayout) -> None   # add widgets; connect their signals
   ↪ to self._changed
  def _load_choices(self, anomaly: LaneAnomaly) -> None   # reset them for a newly selected
   ↪ anomaly
  def _moves(self) -> List[LaneMove]                      # the fix the choices describe, over
   ↪ self._seg
  ```

  The base class turns `_moves()` into the plot preview (yellow segment, magenta rings) and into `pending_fix()`, and only passes on moves that change something.
- **Subclass `FixPanel`** — for a fix that isn't a set of lane moves. Implement `show_anomaly(anomaly, rows)`, `has_pending_fix()`, and `pending_fix()` (returning whatever your type's `apply_fix()` accepts), and emit `preview_changed(Preview | None)` and `pending_changed(bool)` as the choices change.

Return the panel from `create_fix_panel()`. The review panel owns Apply and Undo; a panel never writes to the database.

11.5 Step 4 — register the type, and fixes beyond lane reassignment

Add the type's module to `_BUILTIN_MODULES` in `anomaly_registry.py`. Its position sets the order of the configuration rows and the review buttons. (A type defined outside the package can instead be imported

before the dialogs open; it is listed after the built-ins.) Export the new names from `analysis/lane_anomaly/__init__.py`, except the view module, which is loaded through `UI_MODULE`.

If a fix must change something other than `lane_id`, override `apply_fix()`, `undo_fix()`, and `describe_fix()`, and follow the provenance rules in §3.3:

- register a new `imputed_status` code in `utils.IMPUTED_CODES`, owned by `OWNER_LANE_ANOMALY`, with properties describing what the fix changes (`kinematics_measured=False` if positions or speeds change);
- refuse rows owned by other modules, as `check_correctable()` does;
- save rows with `supersede_rows()` before writing, and restore them with `restore_superseded()` on undo;
- record the fix in `lane_corrections` (with your `FIX_TYPE`) and set the anomaly's status, so the review panel and rerun policy work;
- raise `ValueError` with a user-facing message to refuse a fix; the review panel shows it;
- audit the modules that read codes for the new code.

11.6 Testing a new type

Because `detect()` and the fix functions are Qt-free, test them against a small SQLite database: build an `events` table with the five required columns, call `anomaly_type(KEY).detect(conn, 1, None, {}, 0.1)`, then `ensure_schema(conn)`, `replace_detected()`, and `apply_fix()` / `undo_fix()`, and check `lane_id`, `imputed_status`, and `superseded_points` after each step.

---

12. Connecting Other Data Sources

TRACES currently reads CSV files from the local file system and works on a local SQLite file. This section describes where the code assumes local files, the approaches for supporting other sources — remote URLs, cloud object storage such as Amazon S3 Parquet files, and similar services — and the problems to plan for. It describes a direction rather than an implementation: the right design depends on which services, formats, and security requirements are chosen.

12.1 Where the code assumes local files

Area	Assumption
`gui/import_dialog.py`	The input is chosen with `QFileDialog`; its file filter is built from each configuration's `file_ext`; the output path is the input path with a `.sqlite` extension.
`data_import/csv_process.py`	`check_file()` / `import_data()` open `input_path` with `open()` and measure progress with `os.path.getsize()`. The `BaseProcess` signatures themselves take a `str`, so they don't prevent other sources.
`gui/import_dialog.py` (`_ImportWorkerThread`)	Deletes, creates, and on failure removes the output database as a local file.
`data_import/db_utils.validate_trajectory_db()`	Opens the database with `Path(...).resolve().as_uri()` and `mode=rw`.
`gui/database_session.py`	Opens, closes, copies (SQLite backup API), and switches local database files.
`analysis/base_analysis_worker.py`	Each worker opens its own connection from the database's file path (`PRAGMA database_list`).
`gui/main_window.py`	The recent-files list holds local paths and checks `Path.exists()`.
Export and save dialogs	Write CSV and HTML files to local paths chosen with `QFileDialog`.

SQL throughout the application is SQLite-specific (`PRAGMA table_info`, `INSERT OR IGNORE`, `WITHOUT ROWID`, upserts, triggers).

12.2 Approaches

A. Remote input, local working database (recommended first step). Keep the SQLite database local and treat a remote dataset as another import source. This fits the existing design best:

- Add a `BaseProcess` subclass per format — for example a Parquet processor reading through `pyarrow`, with `fsspec` / `s3fs` (or a provider SDK) for the transport. Parquet is columnar and typed, so `check_configuration()` can map Parquet columns rather than CSV indices, and `import_data()` can read record batches and insert them with `executemany`.
- Put the transport in a small Qt-free layer in `data_import` (it may then be used by `gui`), so processors receive a readable stream or a local cached copy without knowing where it came from. One possible shape:

```
class DataSource(Protocol):          # proposal -- not in the code today
    def describe(self, uri: str) -> SourceInfo        # size, modified time, version tag
    def open_binary(self, uri: str) -> BinaryIO       # streaming read
    def fetch(self, uri: str, local_path: Path, progress) -> None
```

- In `ImportDialog`, let the user enter a URI (or pick from a provider's browser) as well as a local file, and choose the output path explicitly rather than deriving it from the input.
- Report progress in rows or batches when a byte count isn't available.

B. Remote working database. Opening a database that lives in cloud storage is harder, because SQLite needs a local file with working locks and can't operate on object storage directly. The practical pattern is download, work locally, upload:

- `DatabaseSession.open()` downloads the database to a local cache and remembers its origin and version tag (for example an S3 ETag or object version).
- Every module keeps working on the local file unchanged.
- Uploading happens at a defined point — a "Save to Cloud" action or on close — never during a write transaction. Snapshot with the SQLite backup API (as `copy_and_switch()` does) and upload the snapshot.
- `ensure_write_target()` already asks the user where changes go before the first write; it's the natural place to offer "save back to the original location" or "save to a new location".
- The recent-files list, window title, and `validate_trajectory_db()` need to handle URIs.

C. A server database (PostgreSQL, a cloud data warehouse) instead of SQLite would affect nearly every module because of the SQLite-specific SQL, and would need a database abstraction layer first. Treat it as a separate project.

12.3 Authentication and credentials

- **Use each provider's standard credential chain** rather than collecting secrets in TRACES: for AWS, environment variables, shared profiles, and IAM Identity Center (SSO); Azure's `DefaultAzureCredential`; Google Application Default Credentials. A configuration or source entry can name a profile, but never hold a key.
- **Don't store secrets** in JSON configurations, `traces.conf`, recent-file entries, or logs — URIs with embedded tokens (pre-signed URLs) included. If TRACES must remember a credential, use the operating system's credential store (for example via the `keyring` package).
- **Interactive sign-in** (SSO, OAuth device or browser flows) has to work from a desktop application and must not block the GUI thread.
- **Tokens expire.** A long import or upload may outlive a session token; refresh or fail with a clear message, and never leave a half-written database.

- **Read and write permissions differ.** A user may be allowed to read a bucket but not write to it; check before the user spends time making corrections they can't save.
- **Corporate networks** often require proxies and custom certificate authorities; the SDKs and the frozen build need to respect the system's proxy and certificate settings.

12.4 Saving changes back

- **Concurrent edits.** Two users can download the same database. Use conditional uploads (for example S3 `If-Match` on the ETag) or object versioning to detect that the remote copy changed, and offer to save as a new object rather than overwrite.
- **Atomicity.** Upload to a temporary key and rename or copy into place, or rely on the provider's atomic put, so readers never see a partial file. Multipart uploads need to be cleaned up if they fail.
- **Size and time.** Databases can be large; show progress, allow cancelling, and resume where the provider supports it.
- **Offline work.** Decide what happens when the network is unavailable at save time — keep the local copy and retry later, and make its location visible.
- **Provenance.** The row-status and side-table design (§3) already records every correction in the database itself, so a database saved back to the cloud carries its own audit trail.

12.5 Implementation notes

- Do network I/O on worker threads with the existing worker pattern (`log_signal`, `progress_signal`, `interrupt_flag`); never on the GUI thread.
- Keep provider SDK imports inside the modules that use them, so the application starts without them and the lazy package `__init__` files stay light.
- Each SDK is a new dependency: add it to `requirements.txt`, check how PyInstaller bundles it (hidden imports, certificate bundles, plugin entry points such as `fsspec` implementations), and add it to the build's missing-package check (§13).
- Test against local emulators (for example MinIO or `moto` for S3) rather than live services.
- Document new configuration keys in the *TRACES Configuration Guide* and the user-facing workflow in the *TRACES User Guide*.

---

13. Building and Packaging

The Windows build is a GitHub Actions workflow (`build.yml`) that runs on every push:

1. Installs `requirements.txt` and PyInstaller on Python 3.12.
2. Runs `assets/make_icons.py` to build `traces.ico` from `traces_icon.svg` with PyQt6's SVG renderer.
3. Runs PyInstaller in `--onedir` mode with:
   - `--collect-submodules` for `data_import`, `ui_kit`, `analysis`, `plot`, and `gui` — needed because the lazy `__init__` files and the configuration-named processor classes are invisible to PyInstaller's import scan;
   - `--add-data` for `config/`, the two SVGs in `assets/`, and `ui_kit/plot_view.js` (read with `importlib.resources`, so it must sit in a `ui_kit` folder);
   - `--hidden-import` for PyQt6, `PyQt6.QtSvg`, pyqtgraph, superqt, and utm.
4. Fails the build if a bundled data file is missing, or if PyInstaller's warn file lists a required package as missing.
5. Builds an NSIS installer and uploads the portable build and the installer as artifacts.

When you extend TRACES:

- **New modules in existing packages** are collected automatically.
- **A new top-level package** must be added to the `--collect-submodules` list and to the dependency order in §2.

- **A new third-party dependency** goes in `requirements.txt` (pinned), in `--hidden-import` if it's imported lazily, and in the `$packages` pattern of the missing-package check.
- **New data files** need an `--add-data` entry and a line in the bundled-files check, and must be read through `utils.get_resource_path()` or `importlib.resources`.
- **The version** is defined twice — `APP_VERSION` in `ui_kit/brand.py` and in the NSIS script in `build.yml` — and the two must be changed together.

---

14. Cross-Cutting Conventions

- **Worker threads.** Long-running work runs on a worker created and moved to a `QThread` by the GUI-side orchestrator: `_ImportWorkerThread` for imports, `BaseAnalysisWorker` for analyses. Workers open their own SQLite connection, report progress through signals, and check an interrupt flag where cancellation is possible.
- **Computation and persistence are split.** Comp classes compute and return results; dialogs write them, on the main thread, in one transaction, after resolving the write target.
- **Options dicts.** `OPTIONS_SCHEMA` plus an `options` dict keep signatures stable as parameters are added (`BaseComp`, `AnomalyType`).
- **Optional columns.** Gate every read of a non-required column on `utils.events_columns()`.
- **Row filters.** Use `active_rows_clause()` and `measured_rows_clause()` for trajectory data, and status helpers rather than code literals (§3.3).
- **Shared styling.** Use `ui_kit.plot_style` roles and colors, `KinematicStackedPlotWidget`, `BoxPlotGridDialog`, and `plot_export` rather than new plot code.
- **Dialog folders.** Use `utils.last_dialog_dir()` as the starting folder of file dialogs and `remember_dialog_path()` after a choice; name default save files with `default_save_stem()`.
- **Logging.** `logger = logging.getLogger(__name__)` at module scope.
- **Documentation in docstrings.** Module docstrings describe each module's behavior and the reasoning behind it; keep them current when you change behavior.

Appendix: Related Documents

- *TRACES Configuration Guide* — JSON import configurations: column definitions, import functions, units, data dictionary, and reserved names.
- *TRACES User Guide* — end-user workflows, menus, interaction modes, and the correction modules.